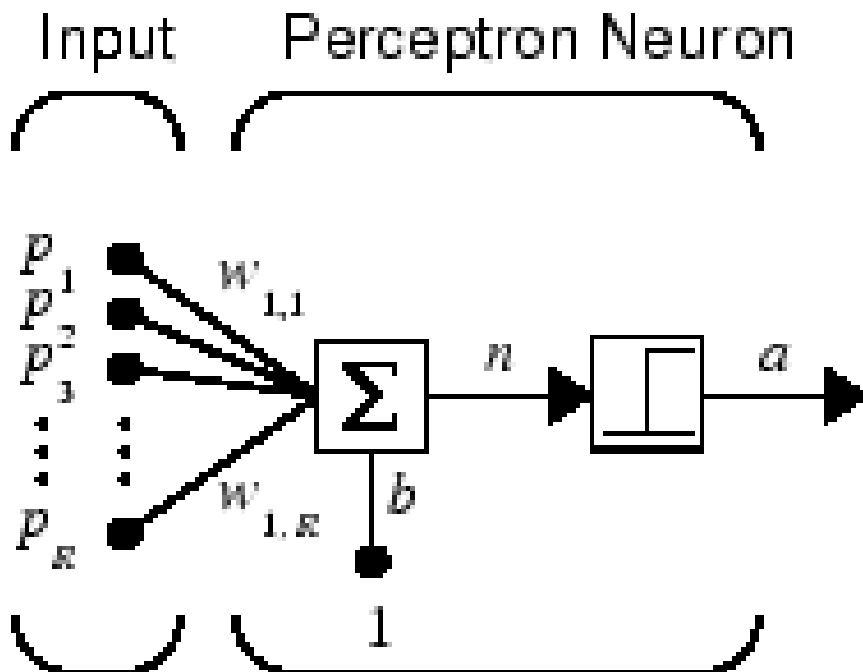
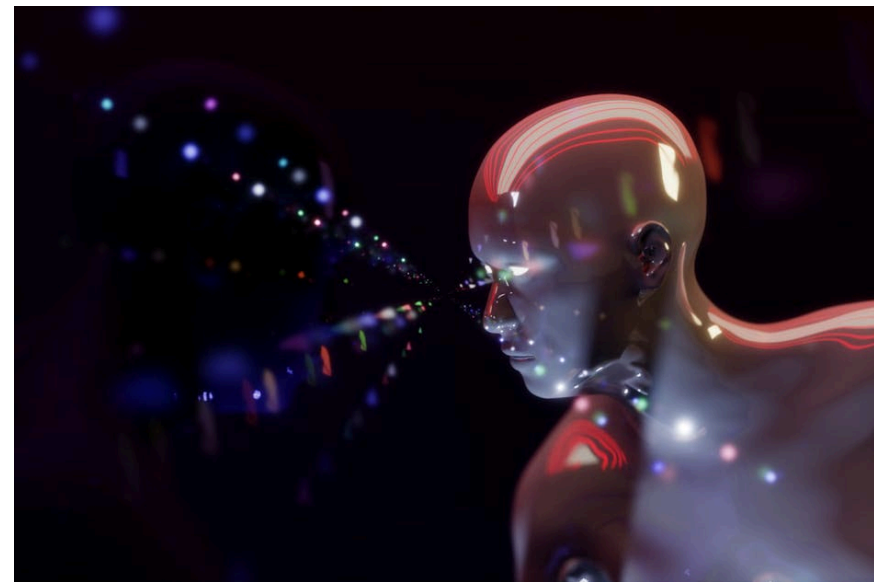


Perceptron Demo Explanations

Dr. Grantham Pang

Sept 2026



$$a = \text{hardlim}(Wp + b)$$

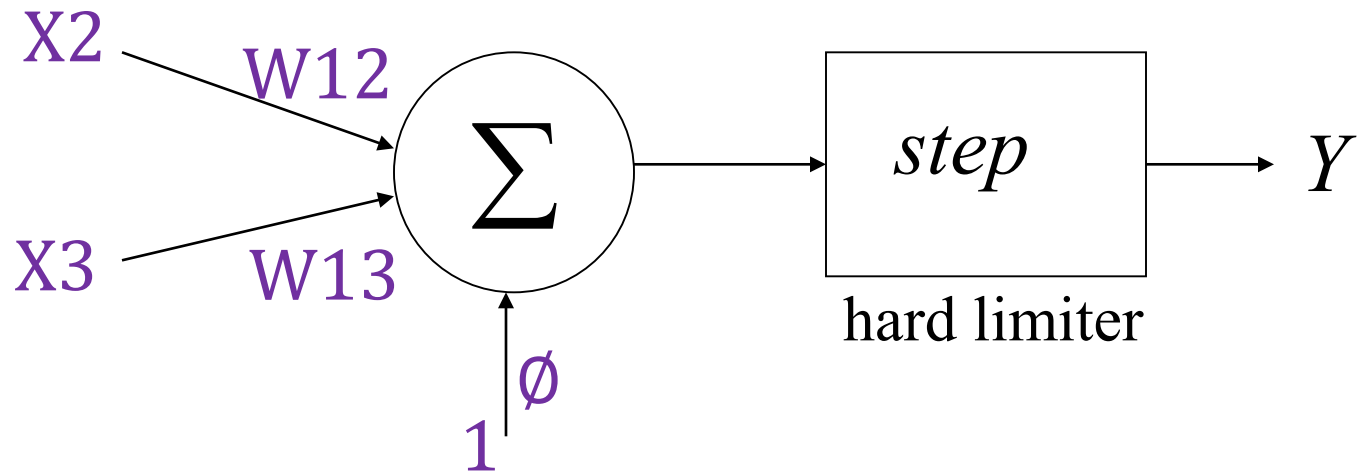
Where

R = number of
elements in
input vector

Model of the Perceptron

Equation:

$$Y(p) = \text{step} \left[\sum_{i=1}^n x_i(p) w_i(p) + \phi \right]$$



The threshold is modeled as an additional input of 1 with weight equals to ϕ .

Explanation on the demo in Python

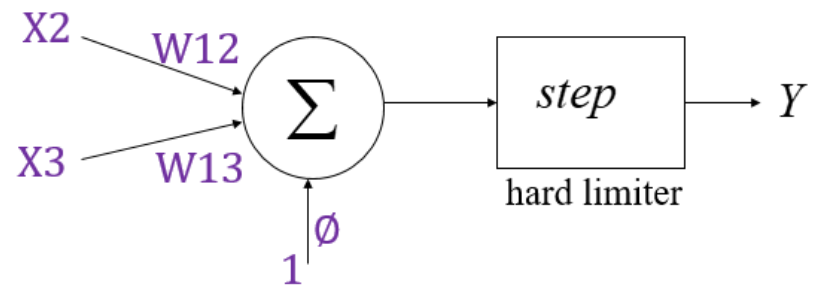
- Filename : **Perceptron_demo.jpynb**
- The perceptron has two inputs (**X2,X3**) and one output. Eight data points (belonging to two classes) are defined. The perceptron is trained to separate the 8 data points. **If the two classes are linearly separable, the perceptron can be obtained.**
- The eight data points in two classes will be displayed on a 2-D plane.
- The equation used in the training algorithm is based on the model of the perceptron.
- The step function is used. Similar result is obtained for the sign function.
- **Ten epochs are carried out. In each epoch, all the eight data points are presented to the perceptron.**
- **W12 is the weight from the input X2 to the perceptron. W13 is the weight from the input X3 to the perceptron.** Threshold is value of theta.
- The convergence to the values of W12, W13 and theta can be observed.

Perceptron Demo Section A

```
In [8]: # Perceptron demo
import numpy as np
import matplotlib.pyplot as plt
import copy
```

Perceptron_demo.jpynb

```
In [2]: # Part 1: Basic Demo of Perceptron
# Perceptron parameters
W12=-3
W13= 2
theta = -1 # the bias term
learnrate = 0.3
```



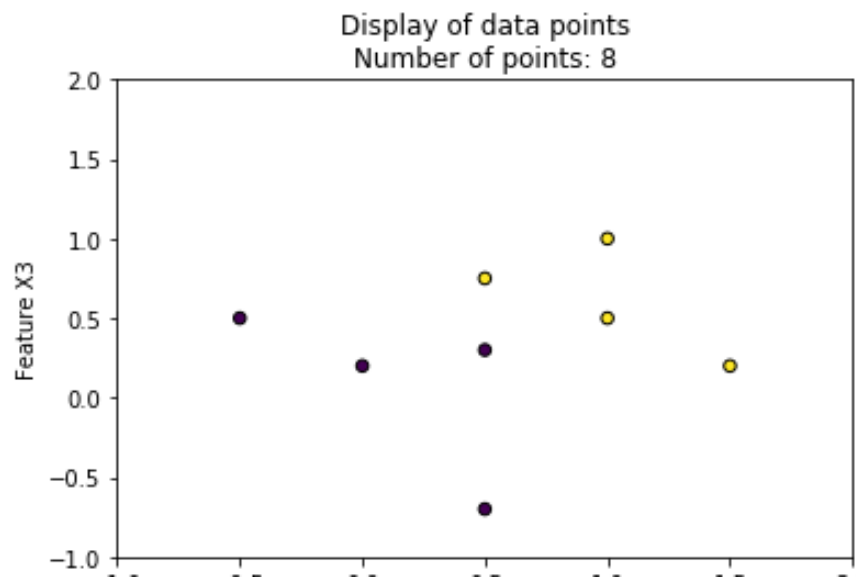
```
# Eight training instances are defined
tmpp= np.array([
    [1.0000 , 1.0000 , 1.0000],
    [1.0000 , 0.5000 , 1.0000],
    [1.5000 , 0.2000 , 1.0000],
    [0.5000 , 0.7500 , 1.0000],
    [0.5000 , 0.3000 , 0.0000],
    [0.5000 , -0.7000 , 0.0000],
    [0.0000 , 0.2 , 0.0000],
    [-0.5 , 0.5 , 0.0000]
])
print(tmpp.shape)
```

(8, 3)

```
In [3]: X2=tmp[(:,0)]
X3=tmp[(:,1)]
desiredout=tmp[(:,2)]
print('X2 = ', X2)
print('X3 = ', X3)
print('desiredout = ',desiredout)

X2 = [ 1.  1.  1.5  0.5  0.5  0.5  0. -0.5]
X3 = [ 1.  0.5  0.2  0.75  0.3 -0.7  0.2  0.5 ]
desiredout = [1. 1. 1. 1. 0. 0. 0. 0.]
```

```
In [4]: # plot
Num_datapoints = len(X2)
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)
plt.title(f'Display of data points\n'
          f'Number of points: {Num_datapoints}')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')
plt.xlim(-1, 2) # xmin=-1, xmax=2
plt.ylim(-1, 2)
plt.show()
```



```
# training algorithm for perceptron (2-dimentional case)
```

```
out = -1
```

```
for iteration in range(10): # i=0,1,2,3,...,9
```

```
    totalerror=0
```

```
    for j in range(8): # j=0,1,2,3,...,7
```

```
        tmpout = W12*X2[j]+W13*X3[j] + theta
```

```
        if (tmpout > 0):
```

```
            out = 1
```

```
        else:
```

```
            out = 0
```

```
        diff = desiredout[j] - out # calculate the error
```

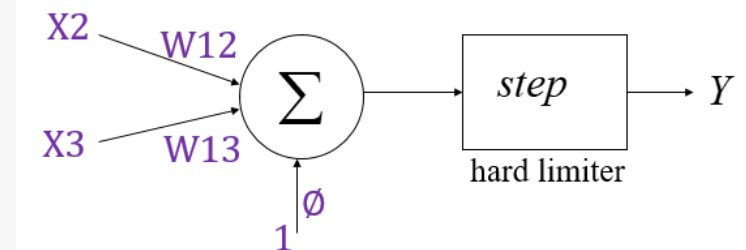
```
        W12 = W12 + learnrate*diff*X2[j]; # error correction learning rule
```

```
        W13 = W13 + learnrate*diff*X3[j];
```

```
        theta = theta + learnrate*diff;
```

```
        totalerror = totalerror + np.abs(diff);
```

```
        #debug print(iteration, j, totalerror, W12, W13, theta)
```



```
# Create meshgrid for decision boundary visualization
xx, yy = np.meshgrid(np.linspace(-1, 2, 100),
                     np.linspace(-1, 2, 100))
print('xx.shape = ', xx.shape)
```

```
xx.shape = (100, 100)
```

```
# Get output for the grid points to plot decision boundaries
percept = copy.deepcopy(xx)

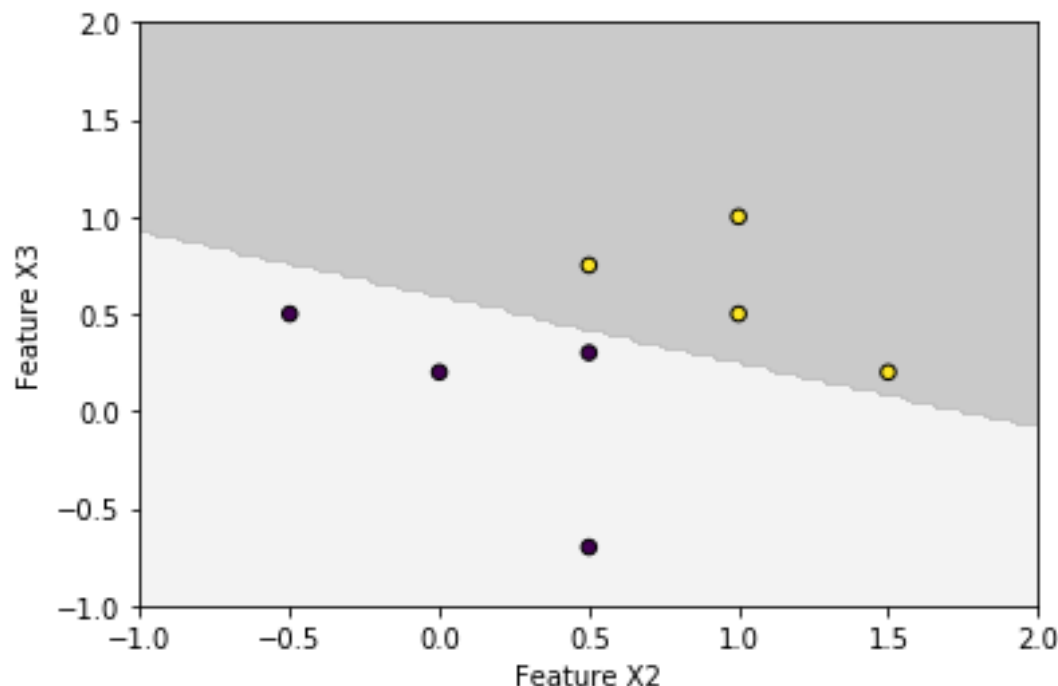
for ixx in range(100):
    for iyy in range(100):
        tmpout=W12*xx[ixx,iyy]+W13*yy[ixx,iyy]+theta
        if (tmpout > 0):
            percept[ixx,iyy] = 1
        else:
            percept[ixx,iyy] = 0
```

```
# Visualize the decision boundary
threshold_1 = 0.5

plt.contourf(xx, yy, percept, levels=[0, threshold_1, 1], alpha=0.3, cmap=plt.cm
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)

plt.title(f'Decision Boundary with threshold {threshold_1}\n')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')
plt.show()
```

Decision Boundary with threshold 0.5



```

# Initialize the vectors to save the values
save_W12 = [-3]
save_W13 = [2]
save_theta = [-1]
save_totalerror = [0]

# Perceptron parameters
W12=-3
W13= 2
theta = -1 # the bias term
learnrate = 0.3

# training algorithm for perceptron (2-dimentional case)
out = -1
for iteration in range(10): # i=0,1,2,3,...,9
    totalerror=0
    for j in range(8): # j=0,1,2,3,...,7
        tmpout = W12*X2[j]+W13*X3[j] + theta
        if (tmpout > 0):
            out = 1
        else:
            out = 0
        diff = desiredout[j] - out # calculate the error
        W12 = W12 + learnrate*diff*X2[j]; # error correction learning rule
        W13 = W13 + learnrate*diff*X3[j];
        theta = theta + learnrate*diff;
        totalerror = totalerror + np.abs(diff);

    save_W12.append(W12)
    save_W13.append(W13)
    save_theta.append(theta)
    save_totalerror.append(totalerror)

```

To save the
parameters
during training

```
# Plotting
print('Length of saved values is ',len(save_W12))
xaxis = list(range(len(save_W12)))

plt.plot(xaxis,save_W13,label = 'W13', marker = 'o')
plt.plot(xaxis,save_W12,label = 'W12', marker = 's')
plt.plot(xaxis,save_theta,label = 'theta', marker = '^')
plt.plot(xaxis,save_totalerror,label = 'totalerror', marker = '*')

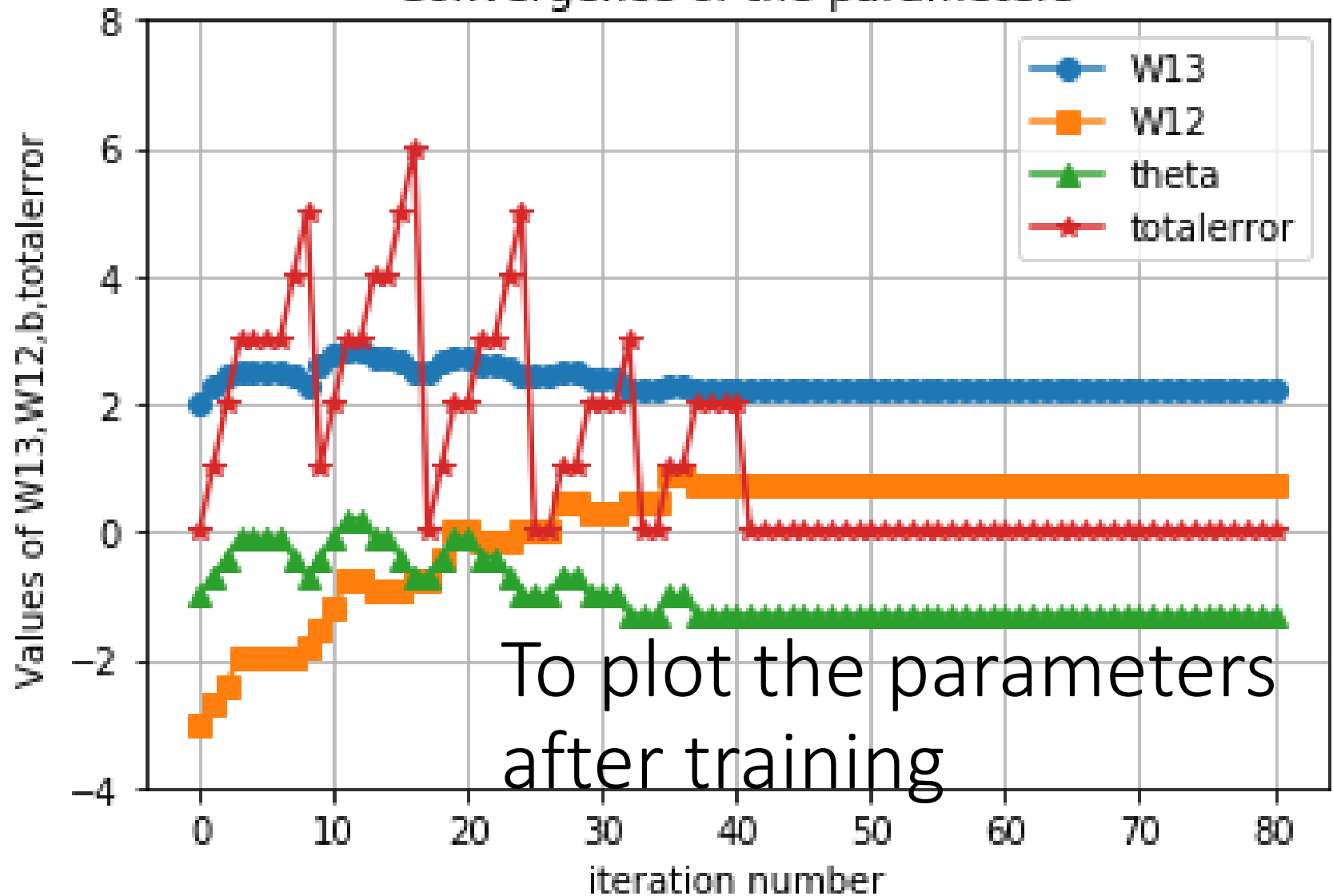
#Add titles and labels
plt.title('Convergence of the parameters')
plt.xlabel('iteration number')
plt.ylabel('Values of W13,W12,b,totalerror')

plt.legend() # To show the legend for the curves
plt.grid()   # To show the grid
plt.ylim(-4, 8)
plt.show()   # To display the plot
```

To plot the
parameters
after training

Length of saved values is 81

Convergence of the parameters



```
# Another simulation with a smaller learning rate
```

```
# -----
```

```
# Initialize the vectors to save the values
```

```
save_W12 = [-3]
```

```
save_W13 = [2]
```

```
save_theta = [-1]
```

```
save_totalerror = [0]
```

```
# Perceptron parameters
```

```
W12=-3
```

```
W13= 2
```

```
theta = -1 # the bias term
```

```
learnrate = 0.16 # the rate has been changed
```

```
# training algorithm for perceptron (2-dimentional case)
```

```
out = -1
```

```
for iteration in range(10): # i=0,1,2,3,...,9
```

```
    totalerror=0
```

```
    for j in range(8): # j=0,1,2,3,...,7
```

```
        tmpout = W12*X2[j]+W13*X3[j] + theta
```

```
        if (tmpout > 0):
```

```
            out = 1
```

```
        else:
```

```
            out = 0
```

```
        diff = desiredout[j] - out # calculate the error
```

```
        W12 = W12 + learnrate*diff*X2[j]; # error correction learning rule
```

```
        W13 = W13 + learnrate*diff*X3[j];
```

```
        theta = theta + learnrate*diff;
```

```
        totalerror = totalerror + np.abs(diff);
```

```
    save_W12.append(W12)
```

```
    save_W13.append(W13)
```

```
    save_theta.append(theta)
```

```
    save_totalerror.append(totalerror)
```

Training using a smaller learning rate

```
# Plotting
print('Length of saved values is ',len(save_W12))
xaxis = list(range(len(save_W12)))

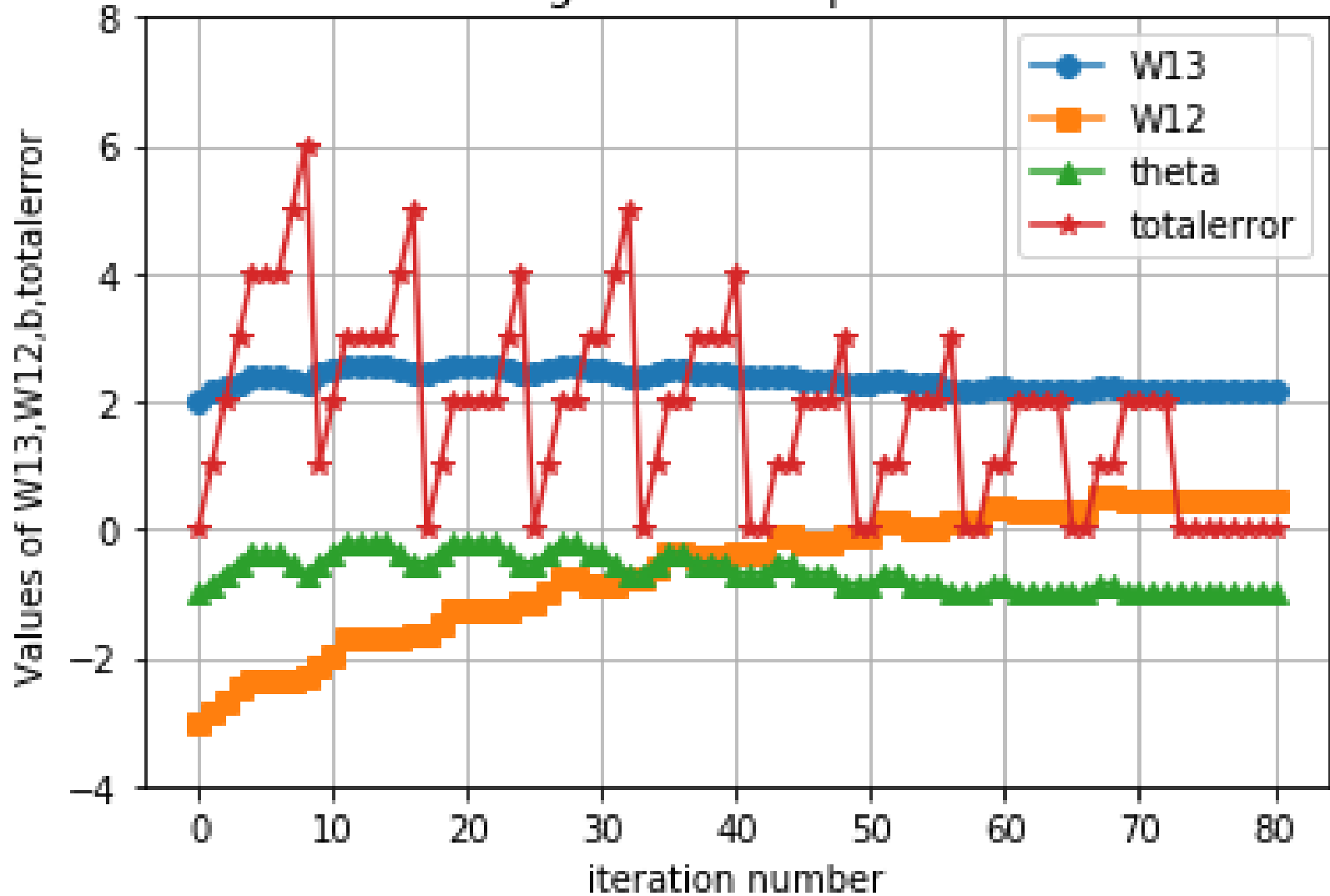
plt.plot(xaxis,save_W13,label = 'W13', marker = 'o')
plt.plot(xaxis,save_W12,label = 'W12', marker = 's')
plt.plot(xaxis,save_theta,label = 'theta', marker = '^')
plt.plot(xaxis,save_totalerror,label = 'totalerror', marker = '*')

#Add titles and labels
plt.title('Convergence of the parameters')
plt.xlabel('iteration number')
plt.ylabel('Values of W13,W12,b,totalerror')

plt.legend() # To show the Legend for the curves
plt.grid()   # To show the grid
plt.ylim(-4, 8)
plt.show()   # To display the plot
```

Takes longer time to settle

Convergence of the parameters



To plot the parameters after training

```

: # Perceptron parameters
W12=-3
W13= -3
theta = -3 # the bias term
learnrate = 0.3

# Eight training instances are defined
tmpp= np.array([
    [-0.7000 ,  1.0000 ,  0.0000],
    [0.0000 ,  0.5000 ,  0.0000],
    [0.5000 , -0.6000 ,  0.0000],
    [-0.5000 ,  0.7500 ,  0.0000],
    [1.5000 ,  0.3000 ,  1.0000],
    [1.8000 , -0.7000 ,  1.0000],
    [0.9000 ,  0.2   ,  1.0000],
    [0.3   ,  1.4   ,  1.0000]
])
print(tmpp.shape)

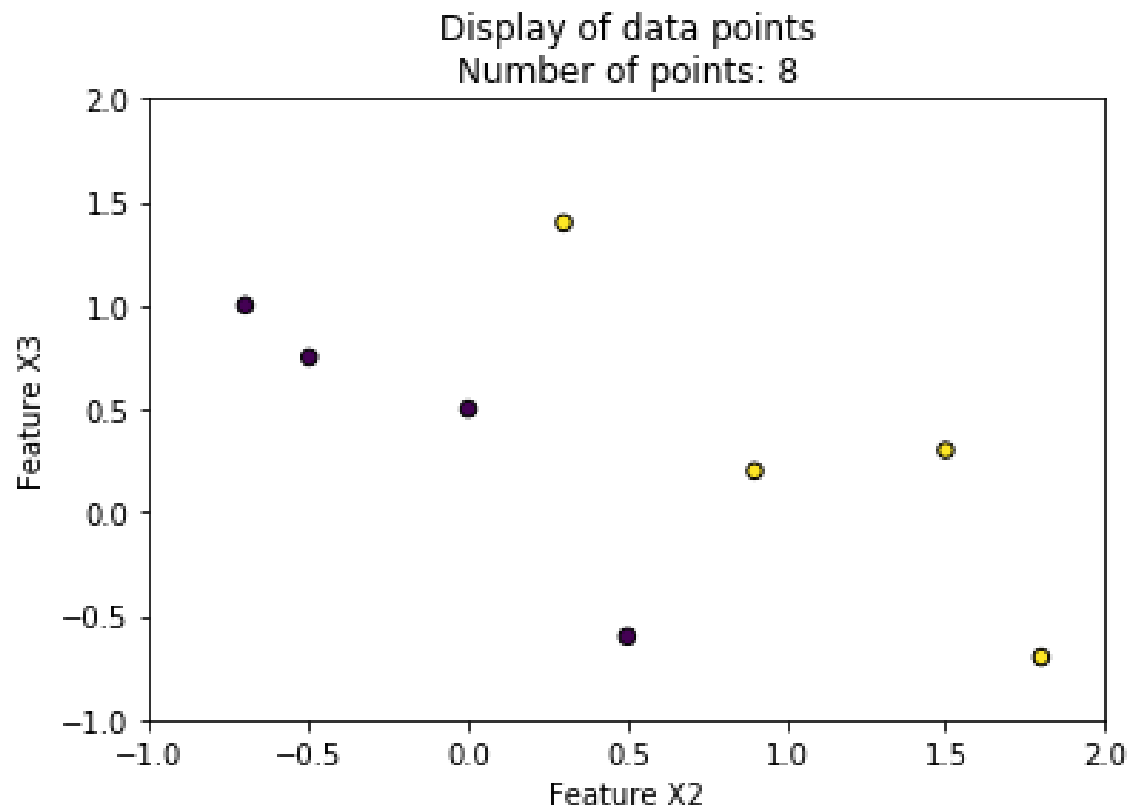
X2=tmpp[:,0]
X3=tmpp[:,1]
desiredout=tmpp[:,2]
#print('X2 = ', X2)
#print('X3 = ', X3)
#print('desiredout = ',desiredout)

```

Training using another
set of data points

(8, 3)

```
# plot
Num_datapoints = len(X2)
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)
plt.title(f'Display of data points\n'
          f'Number of points: {Num_datapoints}')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')
plt.xlim(-1, 2) # xmin=-1, xmax=2
plt.ylim(-1, 2)
plt.show()
```



```

# Initialize the vectors to save the values
save_W12 = [-3]
save_W13 = [-3]
save_theta = [-3]
save_totalerror = [0]

# training algorithm for perceptron (2-dimentional case)
out = -1
for iteration in range(10): # i=0,1,2,3,...,9
    totalerror=0
    for j in range(8): # j=0,1,2,3,...,7
        tmpout = W12*X2[j]+W13*X3[j] + theta
        if (tmpout > 0):
            out = 1
        else:
            out = 0
        diff = desiredout[j] - out # calculate the error
        W12 = W12 + learnrate*diff*X2[j]; # error correction learning rule
        W13 = W13 + learnrate*diff*X3[j];
        theta = theta + learnrate*diff;
        totalerror = totalerror + np.abs(diff);

    save_W12.append(W12)
    save_W13.append(W13)
    save_theta.append(theta)
    save_totalerror.append(totalerror)

```

To save the
parameters
during training

```
# Plotting
print('Length of saved values is ',len(save_W12))
xaxis = list(range(len(save_W12)))

plt.figure(figsize=(8,10))

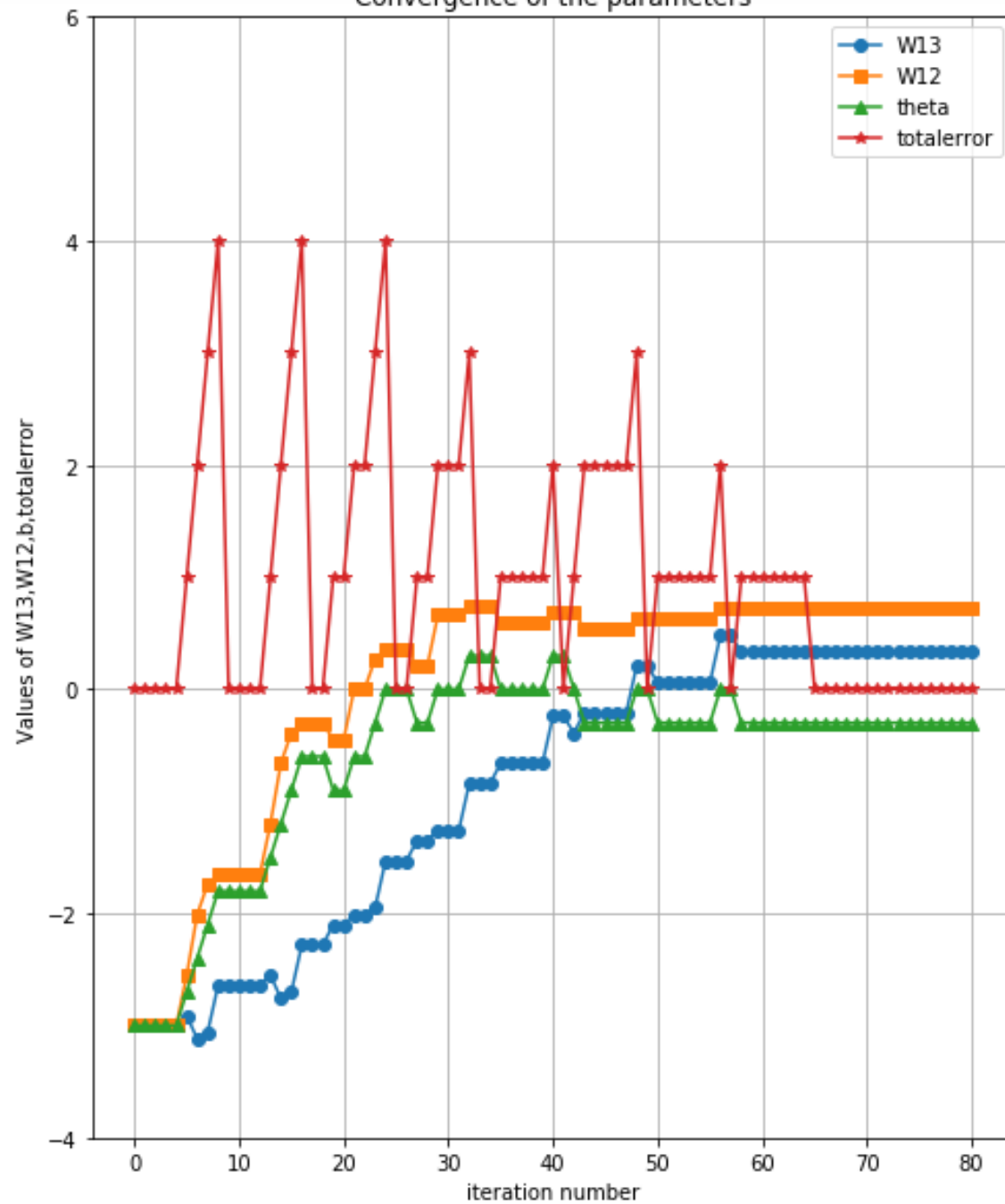
plt.plot(xaxis,save_W13,label = 'W13', marker = 'o')
plt.plot(xaxis,save_W12,label = 'W12', marker = 's')
plt.plot(xaxis,save_theta,label = 'theta', marker = '^')
plt.plot(xaxis,save_totalerror,label = 'totalerror', marker = '*')

#Add titles and labels
plt.title('Convergence of the parameters')
plt.xlabel('iteration number')
plt.ylabel('Values of W13,W12,b,totalerror')

plt.legend() # To show the Legend for the curves
plt.grid()   # To show the grid
plt.ylim(-4, 6)
plt.show()   # To display the plot
```

Length of saved values is 81

Convergence of the parameters



```
# Create meshgrid for decision boundary visualization
xx, yy = np.meshgrid(np.linspace(-1, 2, 100),
                     np.linspace(-1, 2, 100))
print('xx.shape = ', xx.shape)
```

```
xx.shape = (100, 100)
```

```
# Get output for the grid points to plot decision boundaries
percept = copy.deepcopy(xx)

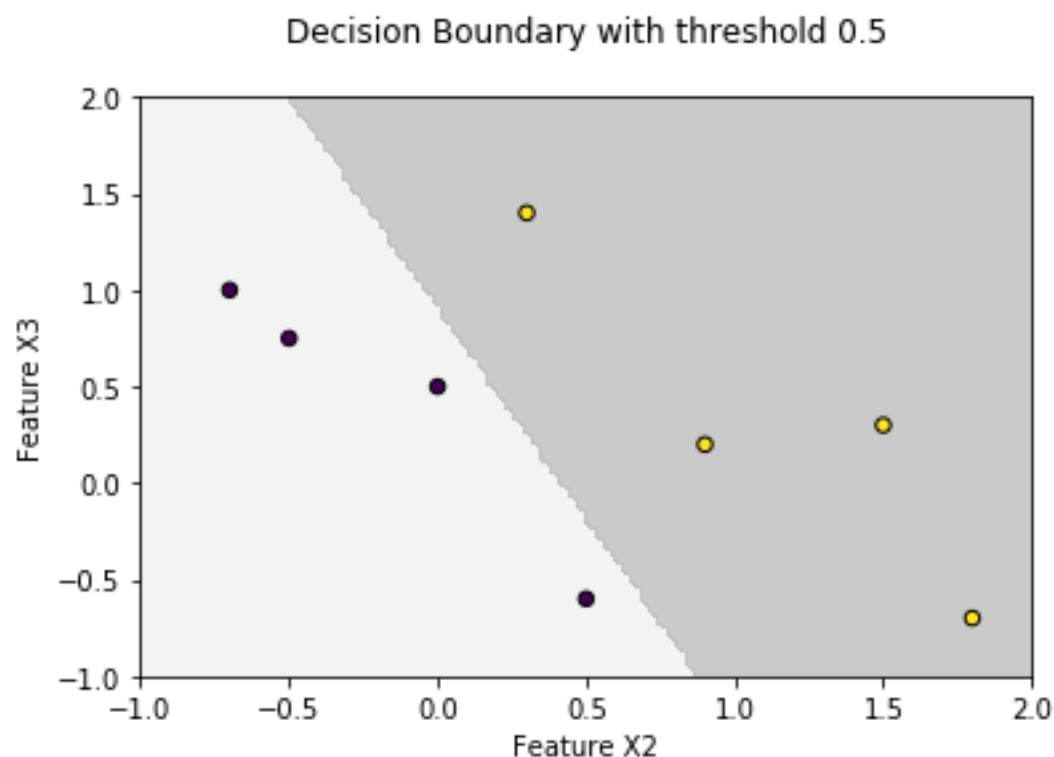
for ixx in range(100):
    for iyy in range(100):
        tmpout=W12*xx[ixx,iyy]+W13*yy[ixx,iyy]+theta
        if (tmpout > 0):
            percept[ixx,iyy] = 1
        else:
            percept[ixx,iyy] = 0
```

```
: # Visualize the decision boundary
threshold_1 = 0.5

plt.contourf(xx, yy, percept, levels=[0, threshold_1, 1], alpha=0.3, cmap=plt.cm
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)

plt.title(f'Decision Boundary with threshold {threshold_1}\n')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')

plt.show()
```



Perceptron Demo Section B

Perceptron_Tutorial_final.jpynb

Color image segmentation

```
import matplotlib.pyplot as plt
import numpy as np
import random
from PIL import Image

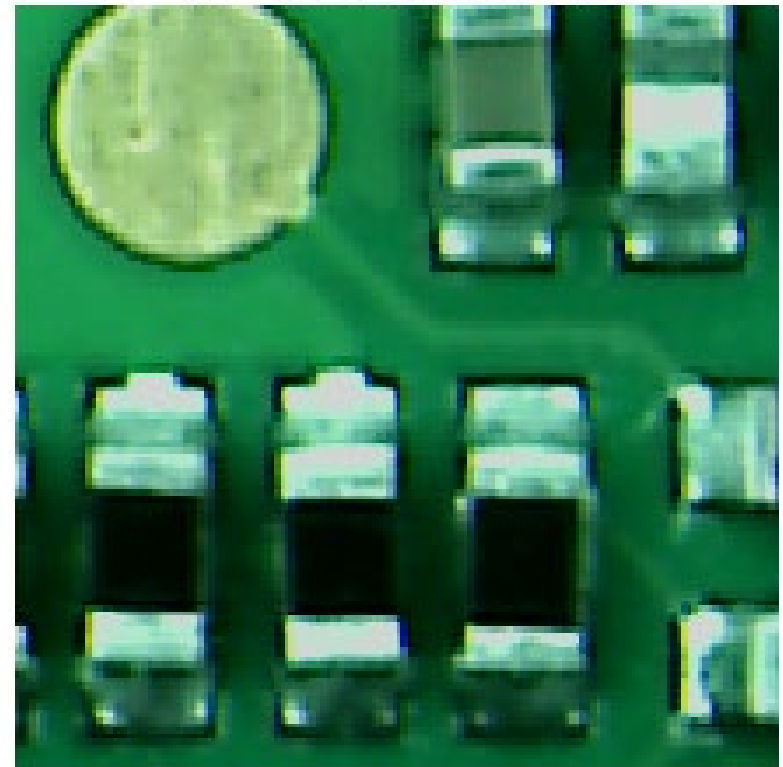
# Open the image file
image_name = "3503A.bmp"
img = Image.open(image_name)

# Ensure the image is in RGB mode
img = img.convert("RGB")

# Get image dimensions
width, height = img.size
print(width,height)

# Display the image
plt.imshow(img)
plt.title("RGB Image Display")
plt.axis('off') # Hide axes |
plt.show()
```

RGB Image Display



```
# original image is called "img", one pixel is shown  
r, g, b = img.getpixel((10, 20))  
print(r, g, b)
```

```
# Another image called "img_new" of the same dimension 150x150 is created  
img_new = Image.new('RGB', (150, 150), color='black')  
pixels = img_new.load()
```

```
# A third image called "img_cluster" of the same dimension 150x150 is created  
img_cluster = Image.new('RGB', (150, 150), color='black')  
pixels_cluster = img_cluster.load()
```

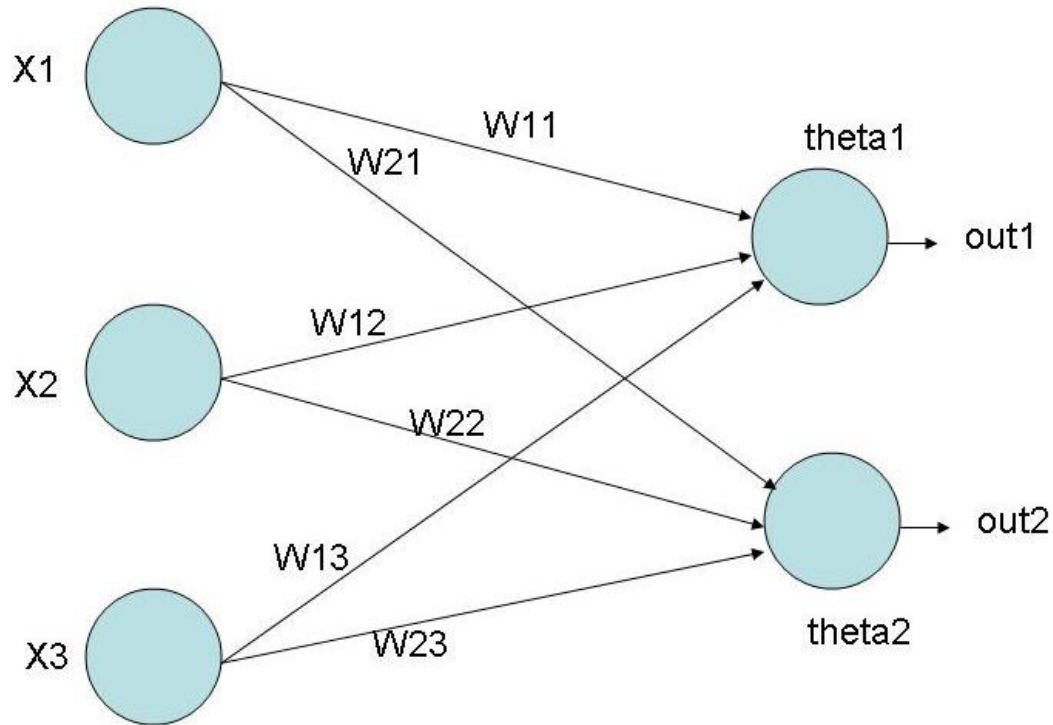
186 224 176

```
#Initial settings
```

```
learnrate = 0.1;
```

```
W11 = random.uniform(-0.8, 0.8)  
W12 = random.uniform(-0.8, 0.8)  
W13 = random.uniform(-0.8, 0.8)  
W21 = random.uniform(-0.8, 0.8)  
W22 = random.uniform(-0.8, 0.8)  
W23 = random.uniform(-0.8, 0.8)  
theta1 = random.uniform(-0.8, 0.8)  
theta2 = random.uniform(-0.8, 0.8)
```

The Perceptron has 3 inputs (R,G,B) and two outputs (4 classes i.e. 4 colors are defined)



```
# Define training instances (for the 4 colors)
```

```
tmpp = np.array([
    [5, 10, 5, 0, 0],      # black
    [0, 96, 50, 255, 0],   # green
    [210, 210, 210, 0, 255], # white
    [0, 20, 0, 0, 0],      # black
    [194, 221, 209, 0, 255], # white
    [205, 215, 210, 0, 255], # white
    [230, 225, 235, 0, 255], # white
    [14, 106, 57, 255, 0],  # green
    [62, 117, 78, 255, 0],  # green
    [26, 156, 81, 255, 0],  # green
    [0, 8, 0, 0, 0],       # black
    [193, 220, 112, 255, 255], # yellow
    [147, 192, 119, 255, 255], # yellow
    [0, 16, 7, 0, 0],      #black
    [177, 221, 156, 255, 255], #yellow
    [159, 204, 126, 255, 255]  #yellow
]) / 255.0
```

```
# Split `tmpp` into inputs (X1, X2, X3) and desired outputs
```

```
X1, X2, X3 = tmpp[:, 0], tmpp[:, 1], tmpp[:, 2]
desiredout1, desiredout2 = tmpp[:, 3], tmpp[:, 4]
print(X1)
```

```
[0.01960784 0.          0.82352941 0.          0.76078431 0.80392157
 0.90196078 0.05490196 0.24313725 0.10196078 0.          0.75686275
 0.57647059 0.          0.69411765 0.62352941]
```

```

# Training loop using the samples
num_iterations = 100
saveWeightsError = np.zeros((num_iterations, 9))

for iterations in range(num_iterations):
    totalerror = 0
    for i in range(16): # Loop over training instances
        # Calculate the first output
        out1 = W11 * X1[i] + W12 * X2[i] + W13 * X3[i] + theta1
        out1 = 1 if out1 > 0 else 0 # Step function
        diff1 = desiredout1[i] - out1

        # Calculate the second output
        out2 = W21 * X1[i] + W22 * X2[i] + W23 * X3[i] + theta2
        out2 = 1 if out2 > 0 else 0 # Step function
        diff2 = desiredout2[i] - out2

        # Update weights and thresholds (perceptron learning rule)
        W11 += learnrate * diff1 * X1[i]
        W12 += learnrate * diff1 * X2[i]
        W13 += learnrate * diff1 * X3[i]
        theta1 += learnrate * diff1

        W21 += learnrate * diff2 * X1[i]
        W22 += learnrate * diff2 * X2[i]
        W23 += learnrate * diff2 * X3[i]
        theta2 += learnrate * diff2

        # Update total error
        totalerror += abs(diff1) + abs(diff2)

    # Save weights, thresholds, and total error for this iteration
    saveWeightsError[iterations, :] = [W11, W12, W13, theta1, W21, W22, W23, theta2, totalerror]

```

```
print("Trained values: ",W11,W12,W13,W21,W22,W23,theta1,theta2)
print(img.size)
```

```
# A python function is defined for finding the distance between
# the image pixel (pixel_v) and the 4 color pixels
# Return the index of the closest pixel
# 0 for black; 1 for white; 2 for green; 3 for yellow
```

```
def dist_function(pixel_v,black_v,white_v,green_v,yellow_v):
    arrp=np.array(pixel_v)
    arrb=np.array(black_v)
    arrw=np.array(white_v)
    arrg=np.array(green_v)
    arry=np.array(yellow_v)
    distb=np.sqrt(np.sum((arrp-arrb)**2))
    distw=np.sqrt(np.sum((arrp-arrw)**2))
    distg=np.sqrt(np.sum((arrp-arrg)**2))
    disty=np.sqrt(np.sum((arrp-arry)**2))

    my_list=[distb,distw,distg,disty]
    min_value=min(my_list)
    min_index=my_list.index(min_value)
    # print(min_value,min_index)

    # 0 is black, 1 is white, 2 is green, 3 is yellow
    return min_index
```

```
Trained values:  -0.0766785936813642  0.5433827698967963  -0.3710254593436008  -0.012455432032343304  0.2
315237493851397  0.5147952297936847  -0.11234697633626967  -0.3914954431333273
(150, 150)
r,g,b =  186 224 176
```

Method 1:

Using Perceptron

Implemented using step

```
# To create a new image
black_v = [0, 0, 0]
white_v = [230, 230, 230]
green_v = [0, 100, 50]
yellow_v = [150, 190, 120]

# Iterate over each pixel in the image
width, height = img.size

for x in range(width):
    for y in range(height):
        # Extract R, G, B values (current pixel) and scaled to [0,1]
        R, G, B = img.getpixel((x, y))
        scaled_R=R/255
        scaled_G=G/255
        scaled_B=B/255

        # Calculate the first output
        out1 = W11 * scaled_R + W12 * scaled_G + W13 * scaled_B + theta1
        out1 = 1 if out1 > 0 else 0

        # Calculate the second output
        out2 = W21 * scaled_R + W22 * scaled_G + W23 * scaled_B + theta2
        out2 = 1 if out2 > 0 else 0

        # Assign colors to "img_new" based on the outputs
        if out1 == 0 and out2 == 0:
            pixels[x,y] = (0,0,0) # Black
        elif out1 == 0 and out2 == 1:
            pixels[x,y] = (255,255,255) # White
        elif out1 == 1 and out2 == 0:
            pixels[x,y] = (0,100,50) # Green
        elif out1 == 1 and out2 == 1:
            pixels[x,y] = (150,190,120) # Yellow
```

```
# find closest index to the colors
pixel_v = [R, G, B]
closest_index = dist_function(pixel_v,black_v,white_v,green_v,yellow_v)
if closest_index == 0:
    pixels_cluster[x,y] = (0,0,0)
elif closest_index == 1:
    pixels_cluster[x,y] = (255,255,255)
elif closest_index == 2:
    pixels_cluster[x,y] = (0,100,50)
elif closest_index == 3:
    pixels_cluster[x,y] = (150,190,120)
```

Method 2: Using Distance measurement

```
# Display the image 1
plt.imshow(img_new)
plt.title("RGB Image Display")
plt.axis('off') # Hide axes for a cleaner image display
plt.show()

print("Trained values: ",W11,W12,W13)
print("Trained values: ",W21,W22,W23)
print("Trained values: ",theta1,theta2)
```

```
# Display the image 2
plt.imshow(img_cluster)
plt.title("RGB Image Display")
plt.axis('off') # Hide axes for a cleaner image display
plt.show()
```

RGB Image Display



Result from
perceptron

RGB Image Display



Result from
distance
measurement

```
save_W11 = saveWeightsError[:,0]
save_W12 = saveWeightsError[:,1]
save_W13 = saveWeightsError[:,2]
save_theta1 = saveWeightsError[:,3]
save_W21 = saveWeightsError[:,4]
save_W22 = saveWeightsError[:,5]
save_W23 = saveWeightsError[:,6]
save_theta2 = saveWeightsError[:,7]
```

```
print(len(save_W11))
```

```
xaxis = list(range(len(save_W11)))
```

```
plt.plot(xaxis,save_W11,label = 'W11', marker = 'o')
plt.plot(xaxis,save_W12,label = 'W12', marker = 's')
plt.plot(xaxis,save_W13,label = 'W13', marker = '^')
plt.plot(xaxis,save_theta1,label = 'theta1', marker = '+')
plt.plot(xaxis,save_W21,label = 'W21', marker = '*')
plt.plot(xaxis,save_W22,label = 'W22', marker = 'v')
plt.plot(xaxis,save_W23,label = 'W23', marker = '<')
plt.plot(xaxis,save_theta2,label = 'theta2', marker = '.')
```

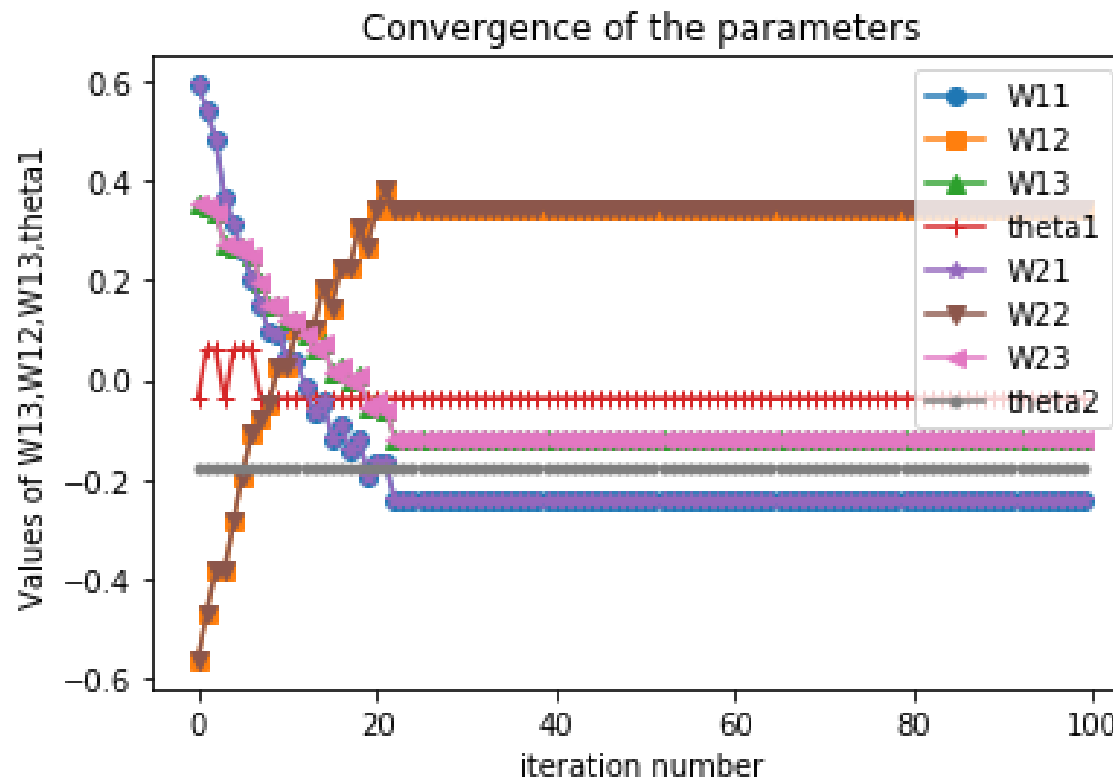
```

#Add titles and labels
plt.title('Convergence of the parameters')
plt.xlabel('iteration number')
plt.ylabel('Values of W13,W12,W13,theta1')

plt.legend() # To show the legend for the curves
#plt.grid()  # To show the grid
#plt.ylim(-4, 8)
plt.show()  # To display the plot

```

100



```

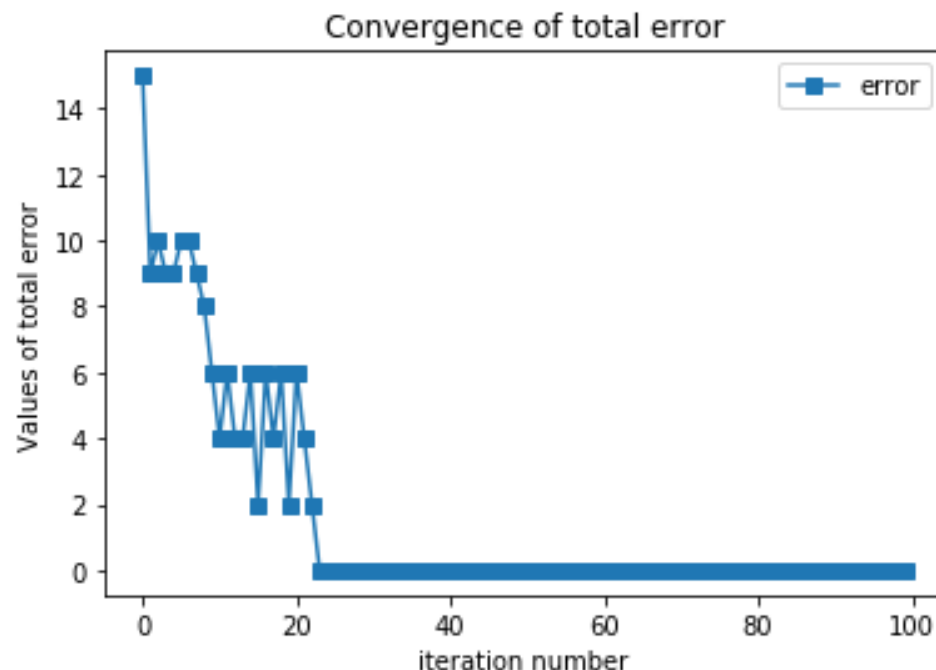
save_error = saveWeightsError[:,8]
print(len(save_error))

xaxis = list(range(len(save_error)))
plt.plot(xaxis,save_error,label = 'error', marker = 's')
#Add titles and labels
plt.title('Convergence of total error')
plt.xlabel('iteration number')
plt.ylabel('Values of total error')

plt.legend() # To show the legend for the curves
#plt.grid() # To show the grid
#plt.ylim(-4, 8)
plt.show() # To display the plot

```

100



```
# Another attempt to implement the perceptron, using sign function
# The code below would save the parameter values during training
# Training loop using the samples
```

```
# new image called "img_sign" is created
```

```
img_sign = Image.new('RGB',(150,150),color='black')
```

```
pixels_sign = img_sign.load()
```

```
learnrate = 0.01;
```

```
num_iterations = 300
```

```
saveWeightsError = np.zeros((num_iterations, 9))
```

```
totalerror = 0
```

```
for iterations in range(num_iterations):
```

```
    totalerror = 0
```

```
    for i in range(16): # Loop over training instances
```

```
        # Calculate the first output
```

```
        out1 = W11 * X1[i] + W12 * X2[i] + W13 * X3[i] + theta1
```

```
        out1 = 1 if out1 > 0 else -1 # Sign function [ changes]
```

```
        diff1 = desiredout1[i] - out1
```

```
        # Calculate the second output
```

```
        out2 = W21 * X1[i] + W22 * X2[i] + W23 * X3[i] + theta2
```

```
        out2 = 1 if out2 > 0 else -1 # Sign function [ changes]
```

```
        diff2 = desiredout2[i] - out2
```

```
        # Update weights and thresholds (perceptron learning rule)
```

```
        W11 += learnrate * diff1 * X1[i]
```

```
        W12 += learnrate * diff1 * X2[i]
```

```
        W13 += learnrate * diff1 * X3[i]
```

```
        theta1 += learnrate * diff1
```

```
        W21 += learnrate * diff2 * X1[i]
```

```
        W22 += learnrate * diff2 * X2[i]
```

```
        W23 += learnrate * diff2 * X3[i]
```

```
        theta2 += learnrate * diff2
```

```
        # Update total error
```

```
        if (abs(diff1) + abs(diff2)) > 0:
```

```
            totalerror += 1
```

Perceptron based
on the "sign" function



```

# To create a new image with pixels_sign
black_v = [0, 0, 0]
white_v = [230, 230, 230]
green_v = [0, 100, 50]
yellow_v = [150, 190, 120]

# Iterate over each pixel in the image
width, height = img.size

for x in range(height):
    for y in range(width):
        # Extract R, G, B values of current pixel, scaled to [0,1]
        R, G, B = img.getpixel((x, y))
        scaled_R = R/255
        scaled_G = G/255
        scaled_B = B/255

        # Calculate the first output
        out1 = W11 * scaled_R + W12 * scaled_G + W13 * scaled_B + theta1
        out1 = 1 if out1 > 0 else -1 #[ changes]

        # Calculate the second output
        out2 = W21 * scaled_R + W22 * scaled_G + W23 * scaled_B + theta2
        out2 = 1 if out2 > 0 else -1 #[ changes]

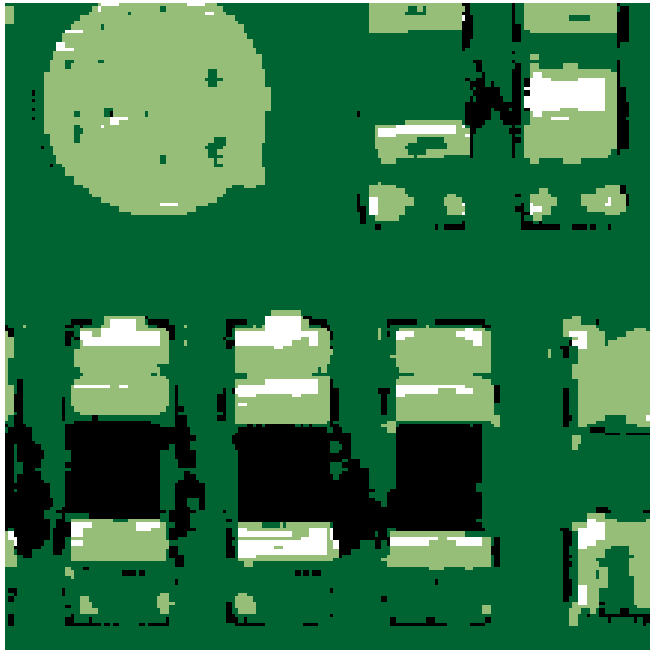
        # Assign colors based on the outputs
        if out1 == -1 and out2 == -1:          #[ changes]
            pixels_sign[x,y] = (0,0,0)        # Black
        elif out1 == -1 and out2 == 1:        #[ changes]
            pixels_sign[x,y] = (255,255,255)   # White
        elif out1 == 1 and out2 == -1:        #[ changes]
            pixels_sign[x,y] = (0,100,50)      # Green
        elif out1 == 1 and out2 == 1:         #[ changes]
            pixels_sign[x,y] = (150,190,120)   # Yellow

```

```
# Display the image img_sign
plt.imshow(img_sign)
plt.title("RGB Image Display")
plt.axis('off') # Hide axes for a cleaner image display
plt.show()

print("Trained values: ",W11,W12,W13)
print("Trained values: ",W21,W22,W23)
print("Trained values: ",theta1,theta2)
```

RGB Image Display



Result from
perceptron
implemented
using sign function